
Práctica 1

Iniciación a la programación en Android y Java

Objetivo: Implementar un juego *endless runner* de scroll horizontal estilo “Jetpack Joyride” para móvil Android y escritorio usando el lenguaje de programación Java.

1. Jetpack Joyride:

Jetpack Joyride es un videojuego de scroll horizontal creado por la compañía Halfbrick Studios. Fue lanzado en la App Store el 1 de septiembre de 2011 y posteriormente en la Google Play en septiembre de 2012. El juego consiste en hacer que Barry (personaje que el jugador controla) avance por el laboratorio y logre hacer la mayor distancia posible. La partida no tiene ningún final, a no ser que Barry muera. A lo largo del laboratorio, hay que esquivar los obstáculos, como electrocutadores, misiles y rayos láser. Mientras más larga sea la distancia lograda, el juego se irá tornando más rápido y complicado. A lo largo de las partidas, hay monedas que pueden ser recogidas por el jugador y usarlas para comprar artículos o utilidades en la tienda, también hay unas cuantas fichas que aparecen en el juego, las cuales sirven para jugar la "lotería final" que aparece cuando la partida se acaba, dicha lotería permite ganar algún premio, como más monedas o alguna ventaja para la próxima partida.





Figura 1: Capturas de pantalla del juego Jetpack Joyride (Google Play).

2. Descripción del juego a implementar:

Pantallas de juego

Al ejecutarse, el juego mostrará una pantalla de bienvenida con el título/logotipo de la aplicación y un “menú” con dos opciones “Jugar” y “Retos” (Figura 2).



Figura 2. Pantalla de título.

Al pulsar el botón de “Jugar” se pasará directamente a la vista de juego principal (Figura 3).



Figura 3: Nivel del juego

El personaje y el mapa

Un nivel de juego consiste en un mapa sin final en el que van apareciendo objetos (como obstáculos o monedas) y en el que según avanza el jugador, aumenta la velocidad de movimiento. El jugador mueve al personaje pulsando en la pantalla, esto hace que el jugador se eleve con un pequeño impulso. El resto del tiempo, el personaje caerá por gravedad.

Para realizar el efecto del mapa infinito, el fondo se compondrá de imágenes que aparecen al final de la pantalla y se mueven de derecha a izquierda. Los obstáculos también aparecerán a la derecha del final de la pantalla y se moverán de derecha a izquierda. Cuando las imágenes de fondo y obstáculos salen por la parte izquierda, se destruirán para liberar memoria.

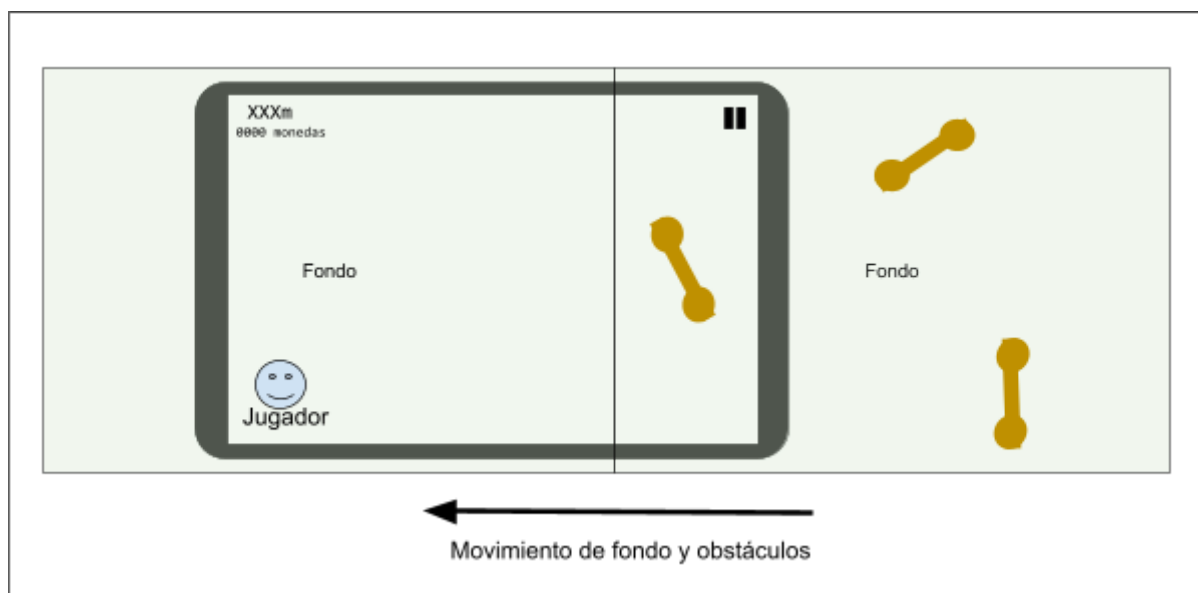


Figura 4: Esquema del nivel de juego

Las monedas y los obstáculos

En los niveles pueden aparecer diferentes tipos de objetos:

- Monedas, el jugador puede obtener y acumular monedas al colisionar con ellas. Por ahora este objeto sirve a modo de puntuación, pero en la próxima práctica tendremos una tienda donde usarla para comprar. Se representan mediante un círculo.
- Zappers, son dispositivos eléctricos fijos o que rotan y que flotan en el nivel. Son el peligro más común del juego. Consisten en dos círculos unidos por un rectángulo.
- Rayos láser, líneas de energía que parpadean o se cruzan en el escenario, obligándote a calcular el momento exacto para posicionarte por encima o por debajo, esquivándolas. Donde van a aparecer estos rayos se indica 2s antes mediante indicadores a ambos lados de la pantalla.
- Misiles, iconos de alerta avisan de su llegada por la derecha de la pantalla 2s antes de que aparezcan. Estos objetos aparecen a la altura indicada por la alerta y se mueven a alta velocidad hacia el lado izquierdo de la pantalla.

Si el jugador colisiona con los zappers, los rayos láser o los misiles, pierde. *Por ahora, todos los objetos y obstáculos estarán representados mediante polígonos sencillos y **no** pintados mediante la carga de imágenes.*

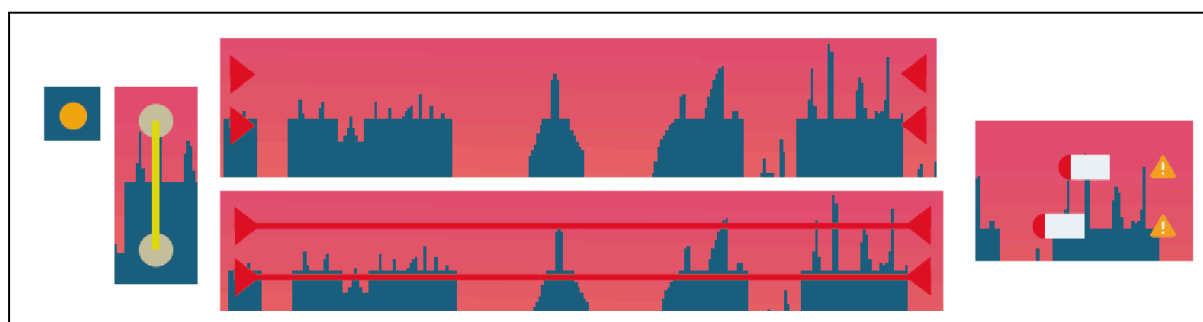


Figura 5: Representación de las diferentes monedas y obstáculos, de izquierda a derecha aparecen: moneda, zapper, rayo láser (indicadores y rayo encendido), misiones y alerta.

Nivel de juego convencional

Al pulsar “Jugar”, se pasará directamente a un nivel de juego. El nivel de juego es infinito y únicamente termina cuando el jugador decide salir o cuando pierde.

En la parte superior derecha habrá un botón de pausa que además de pausar el juego, mostrará dos botones en medio de la pantalla: “continuar” para seguir con el nivel o “salir” para volver al menú.

El nivel de juego comienza a una velocidad “lenta”, de X píxeles por segundo, que irá aumentando según se avanza. La velocidad aumentará de forma lineal cada Y píxeles lógicos recorridos. Podéis definir esa velocidad a vuestro gusto, siempre que tenga sentido dentro del juego.



Figura 6: Vista con el menú de pausa (derecha).

Retos

Al pulsar “Retos” aparecerá una vista con los diferentes niveles de reto que pueden jugarse.



Figura 7: Vista con la lista de retos

Los niveles de reto son niveles de juego especiales con un recorrido y obstáculos predefinidos. A diferencia del juego convencional, estos niveles tienen un máximo de recorrido. La posición de las monedas y obstáculos no será aleatoria y tienen condición de victoria. Los retos pueden ser, por ahora, de 2 tipos según su objetivo:

- Conseguir X cantidad de monedas en Y distancia, el jugador tendrá que conseguir la cantidad de monedas indicada, una vez conseguidas, el nivel se completará con éxito.
- Recorrer X distancia sin perder, el jugador debe recorrer la distancia indicada, una vez conseguido, el nivel se completará con éxito.

Se deben implementar, al menos, 3 retos de cada uno de los tipos, teniendo una dificultad incremental entre aquellos del mismo tipo.

Terminar el nivel

En ambos casos (niveles convencionales) se lanzará un sonido de victoria/derrota y aparecerá la pantalla de Game Over con el mensaje que corresponda y dos botones: “Volver a Jugar” o “Reintentar; y “Volver al Menú”.



Figura 8: Pantalla de derrota para una partida convencional.



Figura 9: Pantalla de derrota para un nivel de reto (izquierda) y pantalla de victoria para un nivel de reto (derecha).

Recomendaciones para lo que será la Práctica 2

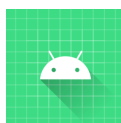
Pensad en que en la práctica 2 extenderemos la funcionalidad del juego, centrándonos en añadir características propias de los dispositivos móviles: anuncios, skins, eventos especiales y más retos, entre otros. Por lo que es recomendable que la representación lógica de los elementos de juego sean lo más genéricas posibles. Por ejemplo, es recomendable que el mapa para los retos tenga una representación que permita configurar

diferentes niveles de forma sencilla. También que los obstáculos y los botones puedan ser representados con figuras geométricas o con sprites.

3. Requisitos de la implementación - *y líneas rojas* -

El juego se implementará en Java, y deberán poderse generar dos versiones diferentes, una para Android y otra para sistemas de escritorio (Windows, GNU/Linux o Mac). El código deberá estar correctamente distribuido en paquetes, ser comprensible y estar suficientemente documentado usando las anotaciones de JavaDoc.

La aplicación en Android debe tener un **icono de aplicación personalizado** y **NO el siguiente**:



Para el desarrollo se hará uso de Android Studio, utilizando un único proyecto con varios módulos. La mayor parte del código deberá ser común y aparecer una única vez, compartido entre ambas versiones. Deberá existir así una capa de abstracción de la plataforma, que se implemente dos veces, una para Android y otra para Escritorio. La implementación del juego deberá hacer uso únicamente de la capa de abstracción y de las funcionalidades del lenguaje que estén disponibles en ambas plataformas. **La parte de lógica y de motor deben estar bien separados** en los módulos correspondientes. Teniendo en cuenta que nuestro motor no es de propósito general y únicamente está para abstraer ambas plataformas en las que trabajaremos (desktop y Android).

La aplicación se debe adaptar a cualquier resolución de pantalla. Independientemente de su relación de aspecto, el juego **no deberá deformarse**, sino aparecer centrado en la pantalla con “bandas” arriba y abajo o a izquierda y derecha. A modo de ejemplo, la siguientes figuras muestran cómo debe verse si se ejecutara en un móvil en vertical con una resolución de 1080 x 2220 (relación 9:18.5) y en una pantalla de ordenador. La lógica espera una relación de 3:2 (por ejemplo 600 x400) por lo que se añaden bandas para que la zona de juego quede centrada.



***Las bandas se muestran en gris oscuro, pero a la hora de realizar la práctica estas deben ser del color del fondo de la pantalla de juego.**

Dado que el punto de entrada de la aplicación es diferente en cada plataforma (en Android se necesita una Activity y en escritorio un main()) se permitirá también la existencia de dos módulos distintos (minimalistas) de “arranque del juego”.

El juego debe estar completo y ser funcional, además debe cumplirse con la entrega en tiempo y forma. Es necesario que **no haya errores en tiempo de compilación** y que el **código esté documentado** adecuadamente para aprobar.

El **repositorio de Github** debe contener un README con la descripción del proyecto y al menos un par de imágenes del juego (menú y nivel). Además debe generarse una release a fecha y hora anteriores de la entrega.

Copiar código de la práctica conlleva el suspenso directo de la asignatura.

4. Consejos y pistas a la hora de implementar

Para abstraer la plataforma, podéis definir los siguientes interfaces:

- **Image:** envuelve una imagen de mapa de bits para ser utilizada a modo de sprite:
 - `int getWidth():` devuelve el ancho de la imagen.
 - `int getHeight():` devuelve el alto de la imagen.
- **Font:** envuelve un tipo de letra para ser utilizado al escribir texto.
- **Graphics:** proporciona las funcionalidades gráficas mínimas sobre la ventana de la aplicación:
 - `Image newImage(String name):` carga una imagen almacenada en el contenedor de recursos de la aplicación a partir de su nombre.
 - `Font newFont(filename, size, isBold):` crea una nueva fuente del tamaño especificado a partir de un fichero `.ttf`. Se indica si se desea o no fuente en negrita.

- `void clear(int color)`: borra el contenido completo de la ventana, rellenándolo con un color recibido como parámetro.
 - Métodos de control de la transformación sobre el canvas (`translate(x,y)`, `scale(x,y)`, `rotate(x,y)`, `save()`, `restore()`). Las operaciones de dibujo se verán afectadas por la transformación establecida.
 - `void drawImage(Image image, ...)`: recibe una imagen y la muestra en la pantalla. Se pueden necesitar diferentes versiones de este método dependiendo de si se permite o no escalar la imagen, si se permite elegir qué porción de la imagen original se muestra, etc.
 - `void setColor(color)`: establece el color a utilizar en las operaciones de dibujo posteriores.
 - `void fillRectangle(cx, cy, width, height)`: dibuja un rectángulo relleno del color activo.
 - `void fillRoundRectangle(cx, cy, width, height, arc)`: dibuja un rectángulo de puntas redondeadas relleno del color activo.
 - `void drawRectangle(cx, cy, width, height)`: dibuja un rectángulo sin relleno y con el borde del color activo.
 - `void drawRoundRectangle(cx, cy, width, height, arc)`: dibuja un rectángulo de puntas redondeadas y con el borde del color activo.
 - `void drawLine(float initX, float initY, float endX, float endY)`: dibuja una línea recta del color activo.
 - `void drawHexagon(float x, float y, float radius)`: dibuja un hexágono regular con tamaño de lado del tercer parámetro recibido.
 - `drawCircle(float cx, float cy, float radius)`: dibuja una circunferencia del color activo.
 - `fillCircle(float cx, float cy, float radius)`: dibuja un círculo relleno del color activo.
 - `void drawText(text, x, y)`: escribe el texto con la fuente y color activos.
- **Input**: proporciona las funcionalidades de entrada básicas. El juego no requiere una interfaz compleja, por lo que se utiliza únicamente la pulsación sobre la pantalla (o click con el ratón).
 - `class TouchEvent`: clase que representa la información de un toque sobre la pantalla (o evento de ratón). Indicará el tipo (pulsación, liberación, desplazamiento), la posición y el identificador del “dedo” (o botón).
 - `List<TouchEvent> getTouchEvents()`: devuelve la lista de eventos recibidos desde la última invocación.
- **Audio**: proporciona la funcionalidad para gestionar y reproducir sonidos.
 - `Sound playSound(string file, boolean loop)`
 - `void stopSound(string file)`
- **Sound**: Envuelve los sonidos
- **Engine**: interfaz que aglutina todo lo demás. Puede ser el encargado de mantener las instancias:
 - `Graphics getGraphics()`: devuelve la instancia del “motor” gráfico.

- `Input getInput():` devuelve la instancia del gestor de entrada.
- `Audio getAudio():` devuelve la instancia del gestor de sonidos

Para separar la lógica de la resolución del dispositivo (o de la ventana) podéis ampliar la clase `Graphics` para que reciba un tamaño lógico (de “canvas”) y que todas las posiciones se den en ese sistema de coordenadas. También podéis plantear el desarrollo de clases adicionales que proporcionen un mayor nivel de abstracción. En particular, para facilitar la puesta en marcha de la aplicación en cada plataforma, es posible que queráis ampliar `Engine` para incorporar la idea de estado de la aplicación.

Al ser interfaces, observad que no se indica cómo se crearán las instancias. Así, por ejemplo, la clase que implemente `Graphics` para la versión de escritorio podría necesitar recibir en su constructor la ventana de la aplicación, y la versión de Android el `SurfaceView` y `AssetManager`. La puesta en marcha de la aplicación tendrá que ser diferente en cada plataforma y estar encapsulada, en la medida de lo posible, en los módulos correspondientes. No obstante, la carga de los recursos no debe programarse dos veces, y debe formar parte del módulo de lógica.

Si en la versión para móvil tenéis problemas de rendimiento, minimizad la creación de objetos (analizad la posibilidad de cachearlos con pools) y usad, si es posible, superficies hardware (`SurfaceHolder::lockHardwareCanvas()`).

5. Opcional

Siempre que los requisitos básicos de la práctica funcionen correctamente y estén bien implementados, se valorará positivamente la incorporación de características adicionales como por ejemplo:

- Fade In - Fade Out de las pantallas al transicionar de una a otra.
- Zappers rotados y/o que roten en tiempo de juego.
- Animaciones para cuando muere el jugador o se consiguen monedas.
- En la versión de escritorio:
 - Inicio del juego en pantalla completa y opción en el menú de cambiar el modo.
 - Pausa del juego (sin consumir recursos) cuando la aplicación pierda el foco.
- Más tipos de retos además de los mencionados en el enunciado

6. Instrucciones de entrega

La práctica debe entregarse utilizando el mecanismo de entregas del campus virtual, no más tarde de la fecha y hora indicadas en el propio campus.

Sólo un miembro del grupo deberá realizar la entrega, que consistirá en un archivo .zip con el proyecto completo de Android Studio eliminando los ficheros temporales. Se añadirá también un fichero **alumnos.txt** con el nombre completo de los alumnos y un pequeño **.pdf** indicando la **arquitectura de clases y módulos de la práctica**. El **documento debe contener una descripción de las partes opcionales y extras desarrolladas**, si ha habido alguna.

7. Evaluación de la práctica

Es obligatorio que la aplicación se ejecute en Android y en Escritorio adaptándose a diferentes resoluciones y manteniendo la relación de aspecto.

A la hora de corregir la práctica se tomará como referencia la configuración y software de los laboratorios.

Los pesos en la evaluación de la práctica serán los siguientes:

- Funcionalidad y código - 80%. En este apartado entran por ejemplo la arquitectura multiplataforma planteada, las funciones creadas, limpieza de código, nombres usados, código comentado, estructuras de datos utilizadas, consumo de recursos....
- Apartado gráfico y experiencia de usuario - 20%.

Los errores en tiempo de ejecución que sean reproducibles descontarán 1.5 puntos. Es importante que probéis y depuréis vuestras prácticas antes de entregarlas.

Una primera evaluación de la práctica se hará usando los laboratorios y de manera individual. Si durante la defensa no se demuestran conocimientos sobre el trabajo presentado el miembro correspondiente del grupo puede obtener menos nota que sus compañeros, llegando a suspender. Aunque os repartáis el trabajo, debéis conocer el proyecto completo (tanto motor como lógica y la arquitectura)

Bibliografía

- Beginning Android Games, Third Edition, Mario Zechner and J. F. DiMarzio, Apress, 2016.
- Developing games in Java, David Brackeen, Bret Barker, Lawrence Vanhelsuwe, New Riders.
- Documentación de Android y Java